Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Bearbeitungszeitraum:	von	20. Januar 2026
	bis	20. Juni 2026

1. Prüfer: Prof. Dr.-Ing. Christian Bergler

2. Prüfer: Prof. Dr. phil. Tatyana Ivanovska

OTH
Amberg-Weiden

Elektrotechnik, Medien
und Informatik

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname
der Studentin/des Studenten: **Weidner, Sebastian**

Studiengang: **Bachelor Künstliche Intelligenz**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: June 13, 2026

Unterschrift:

OTH Amberg-Weiden
Elektrotechnik, Medien und Informatik

Bachelorarbeit Zusammenfassung

Studentin/Student (Name, Vorname):	**Weidner, Sebastian**
Studiengang:	Bachelor Künstliche Intelligenz
Aufgabensteller, Professor:	Prof. Dr.-Ing. Christian Bergler
Durchgeführt in (Firma/Behörde/Hochschule):	OTH Amberg-Weiden
Betreuer in Firma/Behörde:	Prof. Dr.-Ing. Christian Bergler
Ausgabedatum: 20. Januar 2026	Abgabedatum: 20. Juni 2026

Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

---

Zusammenfassung:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Schlüsselwörter: LaTeX, Textsatz, Formeln, Graphiken

Abstract:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Keywords: LaTeX, Textsatz, Formeln, Graphiken

Contents

Acronyms

Symbole, Formelzeichen und Einheiten

Chapter 1

Foundations

1.1 Deep Learning

1.1.1 Neurons

1.1.2 Neural Networks

1.1.3 Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by the structure and functionality of biological brain. They are designed to learn complex relationships from data and have become a fundamental component of modern machine learning. ANNs are capable of approximating highly nonlinear functions and are therefore widely used in applications such as image recognition, natural language processing, forecasting, and control systems.

Neuron

The basic element of an artificial neural network is the artificial neuron. The concept is motivated by biological neurons in the human brain, which communicate through electrochemical signals. In a simplified model, a biological neuron receives signals from other neurons through dendrites, processes the received information, and transmits an output signal through its axon. Artificial neurons mimic this behavior mathematically by combining several input values into a single output.

History. The first mathematical model of an artificial neuron was proposed by McCulloch and Pitts in 1943 [**McCulloch1943ALN**]. Their model consisted of a binary threshold unit that produced an output only when the weighted sum of its inputs exceeded a predefined threshold. Although this model did not include a learning mechanism, it established the foundation for later neural network research.

A major extension of this idea was introduced by Rosenblatt in 1958 with the perceptron [**Rosenblatt1958ThePerceptron**]. In contrast to the McCulloch–Pitts neuron, the perceptron introduced trainable weights and a learning rule, making it possible to adapt the model parameters directly from data. However, the perceptron still relied on a threshold-based activation and was therefore limited to linearly separable classification problems [**Minsky1969Perceptrons**].

This limitation was highlighted by Minsky and Papert, who showed that a single perceptron cannot represent functions such as XOR [**Minsky1969Perceptrons**]. The development of multilayer neural networks together with differentiable activation functions later overcame this restriction. These advances made it possible to train networks with multiple layers and enabled the modern neural network architectures that are used in deep learning today.

Mathematical Formulation. A neuron receives an input vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, where each vector element x_i is associated with a trainable weight w_i from the corresponding weight vector $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$. The neuron first computes a weighted sum of its inputs and adds a bias term b. The resulting pre-activation value is given by:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{1.1}$$

The value z represents a linear combination of the input features. To enable the modeling of non-linear relationships, an activation function $\phi(\cdot)$ is subsequently applied, yielding the neuron's output:

$$y = \phi(z) = \phi\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{1.2}$$

Without the activation function, a composition of multiple neurons would still correspond to a single linear transformation and therefore could not represent complex non-linear patterns. Common activation functions used in modern neural networks include the sigmoid function, the hyperbolic tangent (tanh), and the Rectified Linear Unit (ReLU).

The weighted sum in Equation 1.1 can be expressed more compactly using vector notation and ending in the form of:

$$y = \phi\left(\mathbf{w}^T\mathbf{x} + b\right) = \phi\left(\begin{bmatrix} w_1 & w_2 & \cdots & w_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + b\right) \tag{1.3}$$

This vector formulation serves as the basis for the matrix representation of an entire neural network layer, where multiple neurons are evaluated simultaneously.

Neural Networks

A neural network is a collection of interconnected neurons organized into layers. The most common form of such architectures is the *Multilayer Perceptron* (MLP), also

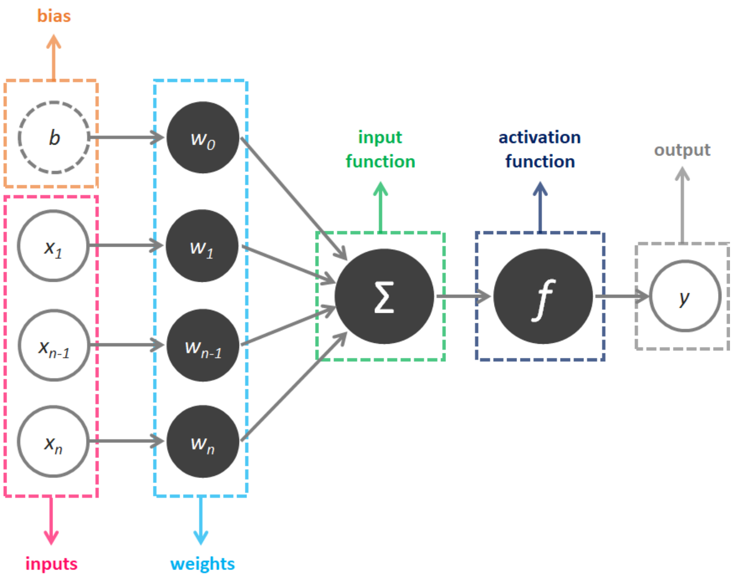

Figure 1.1: Artificial Neuron. Image source: [**xxx**].

referred to as a feedforward neural network, where information flows in one direction from the input layer through one or more hidden layers to the output layer (without forming cycles). The parameters of the network, including the weights and biases, are learned from data using optimization algorithms such as gradient descent. The ability of neural networks to learn complex functions from data has made them a powerful tool for a wide range of applications in machine learning and artificial intelligence.

A neural network consists of multiple interconnected neurons organized into layers The input layer receives the feature vector, while the output layer produces the final prediction. The hidden layers perform intermediate computations that allow the network to learn increasingly abstract representations of the input data.

In a *Multilayer Perceptron Network*, each neuron in a layer receives the outputs of the neurons in the preceding layer as inputs. In a fully connected layer, every neuron is connected to every neuron in the subsequent layer. The output of a layer therefore serves as the input to the next layer.

The computation of an entire layer can be expressed compactly using matrix notation. Let $\mathbf{x}^{(l-1)}$ denote the output vector of the previous layer, $\mathbf{W}^{(l)}$ the weight matrix, and $\mathbf{b}^{(l)}$ the bias vector of layer l. M are the number of neurons in layer l and N the number of neurons in the previous layer $l-1$. The pre-activation vector is given by:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}, \tag{1.4}$$

which can be written explicitly as:

$$\begin{bmatrix} w_{11} & \cdots & w_{1N} \\ w_{21} & \cdots & w_{2N} \\ \vdots & \ddots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{bmatrix}^{(l)} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}^{(l-1)} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_M \end{bmatrix}^{(l)} = \begin{bmatrix} w_{11}x_1 + \cdots + w_{1N}x_N + b_1 \\ w_{21}x_1 + \cdots + w_{2N}x_N + b_2 \\ \vdots \\ w_{M1}x_1 + \cdots + w_{MN}x_N + b_M \end{bmatrix} \tag{1.5}$$

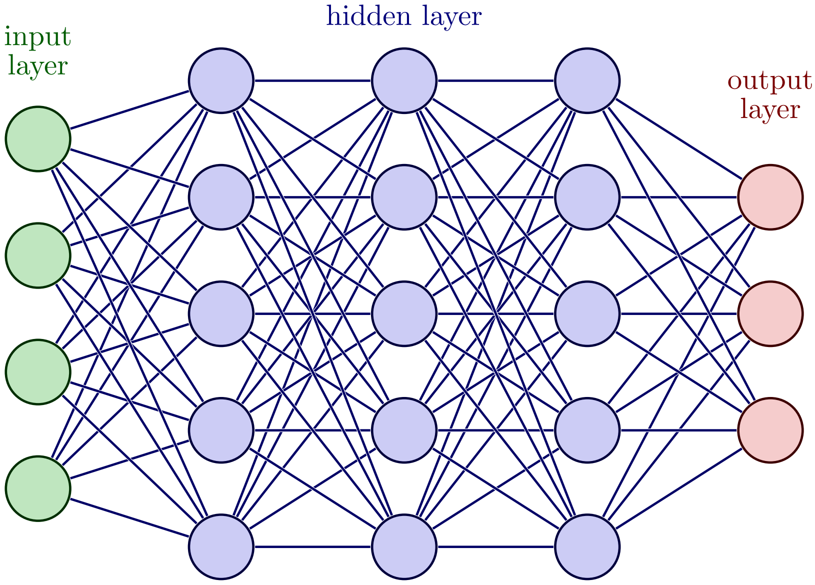

Figure 1.2: Neural Network Architecture. Image source: [**xxx**].

In other words, each neuron in layer l computes its own weighted sum over all outputs of the previous layer and then adds its corresponding bias term.

Equivalently, the bias can be absorbed into the matrix multiplication by augmenting the input vector with a constant entry 1 and extending the weight matrix by one additional column:

$$\begin{bmatrix} z_1^{(l)} \\ z_2^{(l)} \\ \vdots \\ z_M^{(l)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(l)} & \cdots & w_{1N}^{(l)} & b_1^{(l)} \\ w_{21}^{(l)} & \cdots & w_{2N}^{(l)} & b_2^{(l)} \\ \vdots & \ddots & \vdots & \vdots \\ w_{M1}^{(l)} & \cdots & w_{MN}^{(l)} & b_M^{(l)} \end{bmatrix} \begin{bmatrix} x_1^{(l-1)} \\ x_2^{(l-1)} \\ \vdots \\ x_N^{(l-1)} \\ 1 \end{bmatrix} \tag{1.6}$$

Then the activation function is applied element-wise to the resulting pre-activation vector $\mathbf{z}^{(l)}$ to produce the output of layer l:

$$\mathbf{x}^{(l)} = \phi(\mathbf{z}^{(l)}) = \phi\left(\mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}\right). \tag{1.7}$$

Non-Linear Activation Functions in Neural Networks

The introduction of non-linear activation functions is essential, as a composition of purely linear transformations would again result in a linear model. By combining multiple layers with non-linear activations, neural networks can approximate highly complex functions and learn non-linear, intricate patterns from data. In fact, sufficiently large neural networks are universal function approximators, meaning they can approximate a wide range of continuous functions to arbitrary accuracy.

Training Neural Networks

The parameters of a neural network, namely the weights and biases, are learned from data during training. This is typically achieved by minimizing a loss function using gradient-based optimization methods such as gradient descent and its variants. The gradients required for optimization are efficiently computed using the backpropagation algorithm, which propagates the prediction error from the output layer back through the network. This learning process enables neural networks to adapt their internal representations and achieve strong performance across a wide range of machine learning tasks.

—

List of Figures

List of Tables